

Laboration 3

Sorterings algoritmer

Irene Wambui
24irwa@hig.se

27/04/26

Table of Contents

Laboration 2 – Binära sökträd och rekursiva metoder	2
1. Uppgift 1 – Generics	3
2. Uppgift 2 – Binära sökträd	3
2.1 Kort beskrivning av implementationen	4
2.2 Val av krockhantering för dubbletter	4
2.3 Varför finns inte toString i interfacet?	4
2.4 Pseudokod för add	4
2.5 Pseudokod för remove	5
2.6 Kompatibilitet mellan add och remove	5
3. Uppgift 3 – Tidskomplexitet	6
3.1 Mätningar för slumpade heltal	6
3.2 Mätningar för sorterade heltal	7
3.3 Analys	7
4. Slutsats	8

Den här rapporten redovisar en implementation av ett binärt sökträd i Java, motiverar valet av hur dubletter hanteras, beskriver de rekursiva metoderna add och remove samt analyserar uppmätta söktider för både slumpade och sorterade heltal.

1. Uppgift 1 – Generics

1.1 Varför kompilerar inte kodraden `boxTest(new Box<Integer>);`?

Kodraden kompilerar inte eftersom `Box<Integer>` inte är en subklass till `Box<Number>`. Även om `Integer` ärver från `Number` gäller inte samma arv mellan generic-klasser i Java. Metoden `boxTest` förväntar sig ett argument av typen `Box<Number>`, men får istället en `Box<Integer>`, vilket leder till ett typfel vid kompilering.

1.2 Förklaring av `BinarySearchTree<T extends Comparable<? super T>>`

- **T**: är den generiska typen som trädet ska lagra.
- **extends Comparable<...>**: betyder att objekt av typen T måste kunna jämföras med `compareTo`.

Det behövs eftersom ett BST hela tiden avgör om ett nytt värde ska placeras till vänster eller höger.

- **? super T**: betyder att T måste vara jämförbar mot T eller någon supertyp till T. Det gör typen mer flexibel och följer hur `Comparable` ofta deklarerats i Java-bibliotek.

Hela deklarationen betyder alltså att trädet bara får lagra objekt som kan jämföras på ett sätt som är kompatibelt med typen T. Utan detta krav skulle metoder som `add`, `searchFor` och `remove` inte kunna avgöra ordningen mellan elementen.

1.3 Vad innebär PECS-regeln?

PECS står för **Producer Extends, Consumer Super**. Regeln används vid generiska samlingar med wildcard. Om en struktur producerar värden som vi läser från ska man använda `extends`. Om en struktur konsumerar värden som vi stoppar in ska man använda `super`. Exempel: `List<? extends Number>` är bra när man ska läsa `Number`-värden, medan `List<? super Integer>` är bra när man ska lägga in `Integer`-värden.

2. Uppgift 2 – Binära sökträd

Klassen `BinarySearchTree` är implementerad med en inre nodklass `BSTNode`. Varje nod innehåller ett värde item samt två referenser: `left` och `right`. Klassen `BSTNode` behöver inte deklarerats med en egen generic typ eftersom den är en inre klass och därför redan har tillgång till den yttre klassens typ T. Jag använde AI som stöd för att förstå och utforma delar av koden, särskilt den rekursiva metoden `removeRecursive()`. Den slutliga lösningen har jag sedan gått

igenom, anpassat och arbetat vidare med själv för att säkerställa att jag förstår hela implementationen.

2.1 Kort beskrivning av implementationen

- *root* pekar på trädets rot och *size* håller reda på antalet unika element i strukturen.
- *add* och *searchFor* använder *compareTo* för att rekursivt gå vänster eller höger i trädet.
- *toString* traverserar trädet in-order, vilket gör att värdena skrivs ut i sorterad ordning.
- *clear* nollställer *root* och *size*.
- *remove* hanterar de tre klassiska borttagningsfallen: lövnod, nod med ett barn och nod med två barn.

2.2 Val av krockhantering för dubletter

I den här implementationen ignoreras dubletter. Om ett värde redan finns i trädet läggs ingen ny nod till och *size* ökas inte. Det passar bra när trädet ska fungera som en mängd (set) av unika värden. Fördelen är att implementationen blir enkel och att sökning, utskrift och borttagning får ett tydligt beteende: varje värde kan bara förekomma en gång.

2.3 Varför finns inte *toString* i interfacet?

toString behöver normalt inte deklarerats i interfacet eftersom alla Java-klasser redan ärver metoden från *Object*. Man kan ändå skriva en egen implementation för att få en mer användbar strängrepresentation av trädet.

2.4 Pseudokod för *add*

```
add(item)
    om item är null
        kasta IllegalArgumentException

    starta insättningen från roten och spara det uppdaterade trädet i root

addRecursive(current, item)

    om den aktuella noden är null
        öka size med 1
        skapa och returnera en ny nod med item

    om item är mindre än värdet i den aktuella noden
        fortsätt söka rätt plats i vänstra delträdet och spara tillbaka den uppdaterade vänstra grenen

    annars om item är större än värdet i den aktuella noden
        fortsätt söka rätt plats i högra delträdet och spara tillbaka den uppdaterade högra grenen

    annars
        värdet finns redan i trädet
        gör ingenting

    returnera den aktuella noden
```

returnera currentBasfall: current är null, alltså har rätt insättningsplats hittats och en ny nod skapas.

Rekursionssteg: metoden anropar sig själv på vänster eller höger delträd beroende på compareTo.

2.5 Pseudokod för remove

```
remove(itemToRemove)
    om itemToRemove är null eller inte finns i trädet
        returnera false
    starta borttagningen från roten och spara det uppdaterade trädet i root.
    minska size med 1
    returnera true

removeRecursive(current, target):
    om current är null
        returnera null

    om target är mindre än aktuella värdet
        fortsatt söka efter värdet i vänstra delträdet och spara den uppdaterade vänstra grenen
    annars om värdet som ska tas bort är större än värdet i den aktuella noden
        fortsatt söka efter värdet i högra delträdet och spara den uppdaterade högra grenen
    annars
        rätt nod har hittats

    om noden saknar barn
        returnera null
    om noden bara har höger barn
        ersätt noden med dess högra barn och returnera den högra grenen.

    om noden bara har vänster barn
        ersätt noden med dess vänstra barn och returnera den vänstra grenen.

    annars
        hitta den minsta noden i högra delträdet.
        kopiera värdet från den noden till den aktuella noden
        ta sedan bort den nod i högra delträdet som värdet hämtades från och spara tillbaka det
        uppdaterade högra delträdet.
    returnera current
```

Basfall: current är null eller att rätt nod har hittats och något av borttagningsfallen kan utföras direkt.

Rekursionssteg: metoden fortsätter vänster eller höger tills rätt nod hittas. När noden har två barn används in-order-successorn, alltså minsta värdet i höger delträd.

2.6 Kompatibilitet mellan add och remove

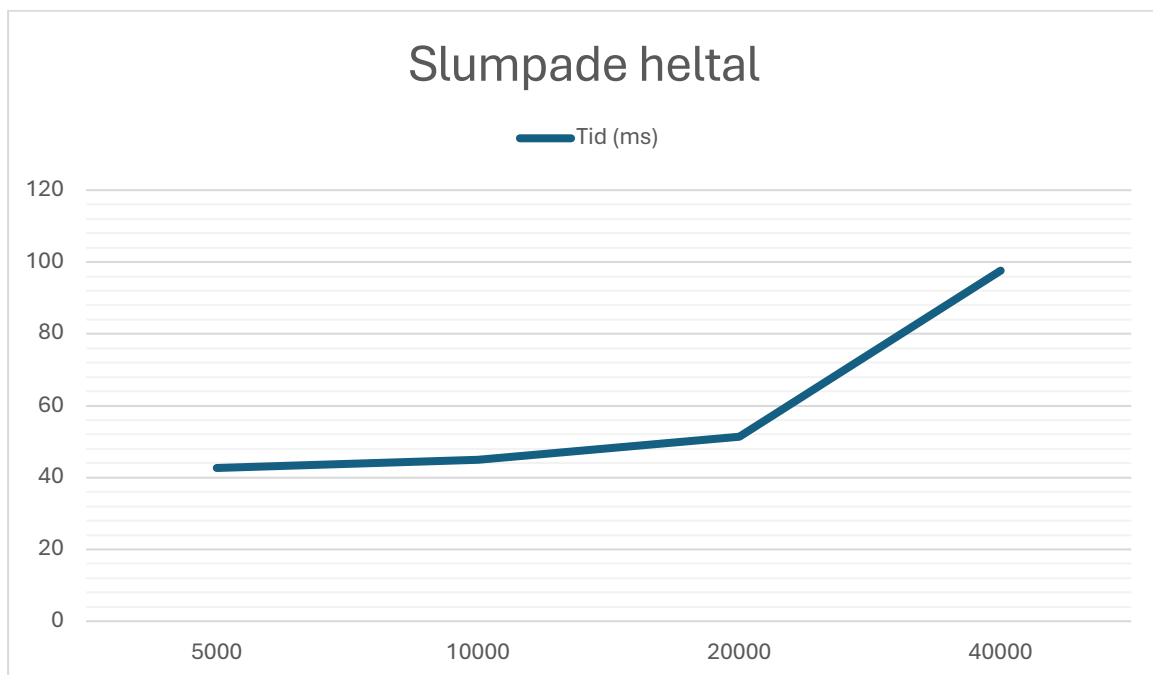
Metoderna *add* och *remove* är kompatibla eftersom båda bygger på att trädet bara innehåller unika värden. *add* skapar aldrig flera noder med samma innehåll och *remove* behöver därför bara ta bort högst en nod för ett givet värde.

3. Uppgift 3 – Tidskomplexitet

Tidstesterna genomfördes genom att bygga träd med olika storlek n och därefter göra 100 000 sökningar per test. För de slumpade träden söktes både värden som fanns och värden som inte fanns i trädet. För de sorterade träden lades heltalen in i stigande ordning, vilket ger ett mycket obalanserat träd.

3.1 Mätningar för slumpade heltal

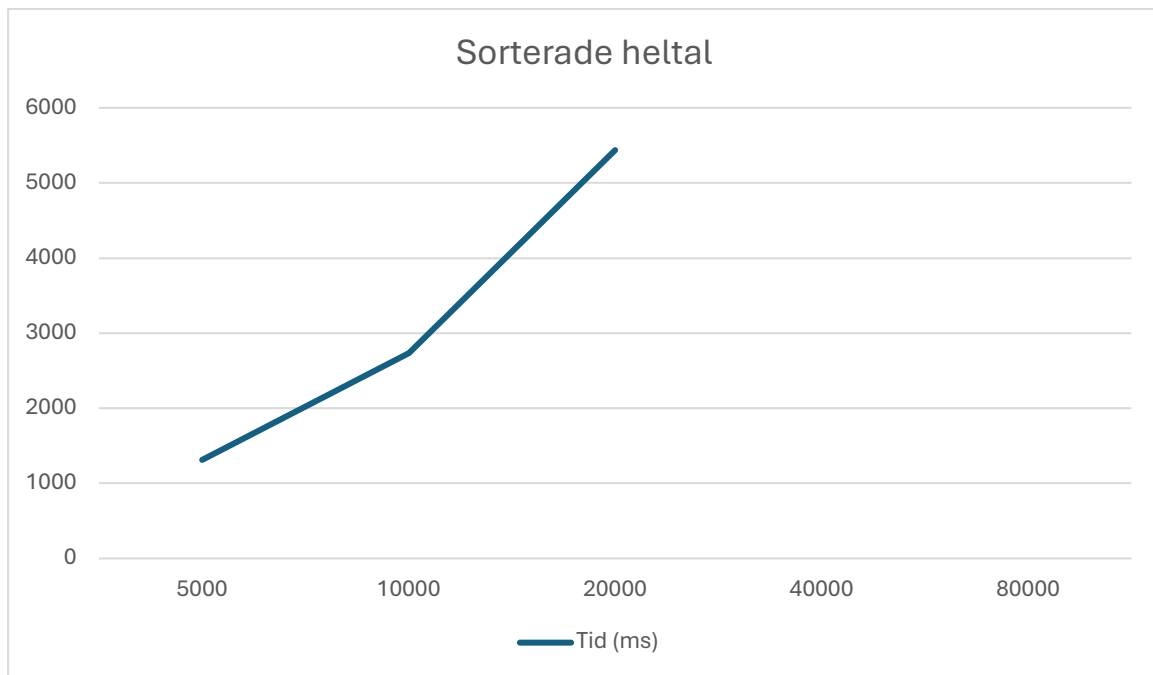
n	Tid (ms)	Kvot $T(2n)/T(n)$
5000	42,663	-
10000	44,927	1.053
20000	51,428	1.145
40000	97,604	1.898
80000	44,049	0.451



Figur 1. Söktider för slumpade heltal.

3.2 Mätningar för sorterade heltal

n	Tid (ms)	Kvot $T(2n)/T(n)$
5000	1312.421	-
10000	2734.987	2.084
20000	5436.959	1.988



Figur 2. Söktider för sorterade heltal.

3.3 Analys

För **slumpade heltal** ökade exekveringstiden långsamt när trädet blev större. Kvoterna låg nära **1**, vilket tyder på att trädet blev relativt balanserat. Detta stämmer med tidskomplexiteten: $O(\log n)$

För **sorterade heltal** ökade exekveringstiden mycket snabbare. Kvoterna låg nära **2**, vilket visar att tiden nästan fördubblades när storleken fördubblades. Trädet blev då obalanserat och liknade en lista. Detta ger tidskomplexiteten: $O(n)$

Vid större test uppstod även `StackOverflowError`, eftersom rekursionen blev för djup i det obalanserade trädet.

4. Slutsats

Implementationen uppfyller laborationens krav med en rekursiv BST, stöd för sökning, insättning, tömning, sorterad utskrift och borttagning. Valet att ignorera dubletter ger ett enkelt och konsekvent beteende. Tidstesterna visar dessutom tydligt den teoretiska skillnaden mellan ett genomsnittligt BST och ett degenererat BST.